Production Data Visibility Path Audit

End-to-end trace of every filter the data passes through, from mobile `POST /site-visits` through mobile completion to the three admin read-paths (Site Visits list, Dashboard, Operations Board). For each endpoint: Prisma where clause, status filter, validationStatus filter, MAINHEAD filter, team filter, tenant filter. All findings are source-only; no app code or DB modified.

HEADLINE — No read-side filter excludes mobile-created SiteVisits. Every ADMIN-mode read path is scoped only by `tenantId`; status and validationStatus filters are no-ops when the admin web does not send them. The "Assets visible, Site Visits / Dashboard / Operations Board empty" pattern cannot be produced by a read-side filter. It can only be produced by (a) no SiteVisit row ever being persisted on production, or (b) tenant mismatch between admin and the row's `tenantId`. The write path has the only filters that can drop a mobile create call — most notably the post-G2 `@IsUUID() mainheadId!` DTO requirement and the team-membership check.

1. Mobile Site Visit creation — POST /site-visits

Server: `SiteVisitsService.create` (`apps/api/src/site-visits/site-visits.service.ts:375–496`). Mobile call site: `apps/mobile/src/api.ts:351–381`; payload assembled at `CheckInScreen.tsx:551–564`.

Write filters (validation gates before `prisma.siteVisit.create`):

Filter	Location	Behaviour
Tenant	`site-visits.service.ts:436`	Row written with `tenantId: user.tenantId`. The mobile user's tenant becomes the SiteVisit's tenant. Cross-tenant access by admin is impossible without a matching tenant on the admin side.
Team existence	`site-visits.service.ts:378–391`	`NotFound` if `dto.teamId` is not an active team within the user's tenant.
Team membership (non-admin)	`site-visits.service.ts:393–411`	Non-ADMIN users must be active members of the team. Otherwise `Forbidden`. Mobile pilot users are typically TECHNICIAN — must be team-bound.
MAINHEAD — DTO (G2)	`create-site-visit.dto.ts:45–46`	**Post-G2 strict requirement:** `@IsUUID() mainheadId!: string`. Missing or non-UUID → 400. Pre-G1 visits in DB will have `mainheadId=null`; new pilot creates can no longer save without it.
MAINHEAD — service (G1)	`site-visits.service.ts:1104`	New-pencawang check-in: `if (!dto.mainheadId) push missing`. The DTO already gates this; this is defense-in-depth.
GPS / pencawang	`site-visits.service.ts:1100–1121`	New-pencawang flow requires Kod / Nama / FunctionalLocation / GPS coordinates / GPS accuracy. Missing fields → 400.
Substation conflict	`site-visits.service.ts:486–495`	Prisma P2002 → 409 "Pencawang with this code already exists."

Row written (success case):

```
status            = normalizeCreateStatus(dto.status) → ACTIVE by default
                  (only OPEN or IN_PROGRESS overrides; mobile never sets it)
validationStatus = PENDING
completedAt     = null  (not set at create-time)
mainheadId      = dto.mainheadId  (G2-required)
tenantId        = user.tenantId  (mobile user's tenant)
teamId          = dto.teamId
```

2. Mobile Visit Completion — POST /site-visits/:id/complete

Server: `SiteVisitsService.complete` (`site-visits.service.ts:716–747`).

Pre-update filters:

Filter	Location	Behaviour
Mutate authority	`site-visits.service.ts:717`	ADMIN / MANAGER / SUPERVISOR / TECHNICIAN allowed. VIEWER / CLIENT blocked.
Accessibility	`findAccessibleSiteVisit (~line 1124)`	Same as list `accessScope`: tenant + (for non-admin) active team membership.
Mutable status	`assertVisitIsMutable`	Throws if visit is in a terminal status (COMPLETED, CANCELLED).
Asset rollup	`validateCompletion + materializeImplicitVisitAssetLinks`	Synchronises asset linkage and checks the completion rules (assets present, no pending inspections).

Row updated on success:

```
status           = COMPLETED
completedAt    = dto.completedAt ?? new Date()
endedAt        = same as completedAt
validationStatus = PENDING  (reset on completion — visit re-enters QA queue)
completionNotes = dto.completionNotes (if supplied)
```

Note: completion does NOT delete the row, does NOT change tenant or team, and does NOT change MAINHEAD. The only fields touched are status, `completedAt`, `endedAt`, `validationStatus`, and optional notes.

3. Site Visits list — GET /site-visits

Server: `SiteVisitsService.list` (`site-visits.service.ts:498–523`), `buildListWhere` at lines 1812–1839, `accessScope` at lines 2136–2151. Admin web call: `apps/admin-web/src/lib/site-visits.ts:497` — `GET /site-visits` with NO query parameters.

Filter	Behaviour given the admin call
Tenant	Always applied: `tenantId: user.tenantId`. ADMIN sees only their own tenant's rows.
Access scope	ADMIN: {} (no extra filter). Non-ADMIN: `team.members.some.userId = user.id AND isActive=true`.
Status	Admin web sends no `status` query param → {}. All statuses returned (ACTIVE, OPEN, IN_PROGRESS, COMPLETED, CANCELLED). When `status=ACTIVE` IS provided (mobile uses this), it expands to `status IN [ACTIVE, OPEN, IN_PROGRESS]`.
ValidationStatus	Admin web sends none → {}. All validation statuses returned including PENDING.
MAINHEAD	Optional `mainhead` query param (text search on legacy text column). Admin web does not send it for the list page. → {}
Team	Optional `teamId` query param. Admin web does not send it. → {}
Other optional filters	`visitType, operationalDomain, operationMode, operationalScope, sessionKind, userId, pencawang, dateFrom, dateTo, search` — all skipped when not provided.

Effective ADMIN query: `WHERE tenantId = adminTenantId`. Nothing else. **If admin sees zero visits, the row count for the admin's tenant is zero.**

4. Dashboard — GET /dashboard

Server: `DashboardService.getDashboard` (`apps/api/src/dashboard/dashboard.service.ts:35–`). Builds N parallel queries against SiteVisit, Asset, Inspection, Defect.

Sub-query	Where
Asset count	`accessibleAssetWhere = { tenantId: user.tenantId }`. **No team scope.** ADMIN, QA Manager, and TECHNICIAN all see the same asset count for their tenant. This is why assets appear when nothing else does.
Inspection count	`{ tenantId, …inspectionAccessScope }`. `inspectionAccessScope` = `{}` for ADMIN; `{ siteVisit: { team: { members: { some: { userId, isActive } } } } }` for non-ADMIN.
Defect groupBy status / severity	`accessibleDefectWhere = { inspectionItemResult: { isDefect: true, inspection: accessibleInspectionWhere } }`. Inherits tenant + non-admin team scope.
Active visit count	`{ tenantId, …siteVisitAccessScope, status: { in: [ACTIVE, OPEN, IN_PROGRESS] } }`.
Completed visit count	`{ tenantId, …siteVisitAccessScope, status: COMPLETED }`.
SiteVisit groupBy status / validationStatus / visitType	Same base where: `accessibleSiteVisitWhere = { tenantId, …siteVisitAccessScope }`. No status pre-filter on the groupBy.
Defect overdue / SLA queries	Inherit `accessibleDefectWhere` + add `status: { in: ACTIVE_SLA_STATUSES }` and `dueDate < now`.

For ADMIN: every SiteVisit-related count is scoped by tenantId only. If admin sees 0 across all those counts but a positive Asset count, that is fully consistent with Assets existing on the tenant but no SiteVisit rows existing on the same tenant.

5. Operations Board — GET /defects/operations-board

Server: `DefectsService.getOperationsBoard` (`defects.service.ts:563–`); `buildOperationsBoardWhere` at lines 1743–1772; `inspectionAccessScope` at lines 3144–3161.

Filter	Behaviour
Defect → defectiveness	Always: `inspectionItemResult.isDefect = true`. Only inspection results flagged as defects appear.
Tenant	Always: `inspection.tenantId = user.tenantId`.
Access scope	ADMIN: `{}`. Non-ADMIN: `inspection.siteVisit.team.members.some.userId = user.id`. **This is the QA Manager wall** — QA Managers (role=MANAGER) are not team members, so they see zero.
Status	Optional `status` query → no filter when not provided.
ValidationStatus	Not a defect-side filter. Visit-level validationStatus is not consulted by the Operations Board.
MAINHEAD	Optional `mainhead` query param. Admin web does not pre-set it. → `{}`
Team	No direct team filter from query — the non-ADMIN team-membership requirement is the only team gate. → `{}`

Operations Board emptiness for ADMIN means there are no defects whose inspection is in admin's tenant. Defects only exist if an inspection has been submitted AND an inspection item was flagged as defective. Zero visits → zero inspections → zero defects → empty Operations Board (always).

6. Specific verifications

6.1 ACTIVE vs OPEN / IN_PROGRESS / COMPLETED compatibility

- Mobile create writes `status=ACTIVE` by default. Mobile completion writes `status=COMPLETED`. The ACTIVE_SITE_VISIT_STATUSES helper (`site-visits.service.ts:44–48`) treats `{ ACTIVE, OPEN, IN_PROGRESS }` as interchangeable "active" — so an ACTIVE row IS counted by Dashboard's "active visit count" and is returned by the list when `?status=ACTIVE` is queried.
- The admin web Site Visits page does not send any status filter, so all statuses are returned. A completed ACTIVE-flow visit will still appear (with `status=COMPLETED`) after Complete Visit.
- **No compatibility gap. The ACTIVE / OPEN / IN_PROGRESS family is handled uniformly by every read path on the admin side.**

6.2 validationStatus=PENDING visibility

- `PENDING` is the create-time default and the post-completion default. Mobile flow always produces PENDING rows.
- Admin web list does not filter by validationStatus by default. PENDING rows are returned. The list page allows the admin to optionally filter by validation status, but no such filter is applied at page load.
- Dashboard counts PENDING rows in `siteVisitValidationCounts` and in all visit totals. PENDING is not excluded.
- **No PENDING visibility gap on the admin read side.**

6.3 completedAt requirements

- Mobile-created (un-completed) visits have `completedAt=null`. Admin web list returns them; Dashboard counts them as active.
- After mobile completion, `completedAt` is set to the supplied or current timestamp. Visit moves to `status=COMPLETED` and is counted in "Completed visit count".
- **No completedAt-based exclusion anywhere**. The dashboard does NOT require `completedAt IS NOT NULL`.

6.4 MAINHEAD requirements after G1 / G2

- G2 hardened the CREATE side: `create-site-visit.dto.ts:46` now declares `mainheadId!: string` with `@IsUUID()` only — `@IsOptional()` removed. A POST that omits or mistypes `mainheadId` is rejected with 400.
- G2 did NOT add any MAINHEAD filter to the READ side. Site Visits list, Dashboard, Operations Board all leave `mainheadId` unconstrained unless the admin explicitly filters.
- Pre-G1 visits already in the DB with `mainheadId=null` remain visible — the legacy text column `SiteVisit.mainhead` is still rendered by the detail page with a "Legacy MAINHEAD" badge per the Site Visit cleanup.
- **MAINHEAD does not exclude rows from any admin read path.** The only MAINHEAD gate in the data path is at create-time, which can prevent rows from existing in the first place.

7. Are mobile-created SiteVisits excluded by any filter?

READ side: no. For ADMIN, every read path is `tenantId`-only. For non-ADMIN (including QA Manager), the team-membership scope drops every visit the user is not in — but this is an access decision, not a row-level exclusion of the data itself.

WRITE side: yes, several gates can prevent rows from being written at all. In order of post-pilot likelihood:

Gate	Resulting symptom on admin
G2 `@IsUUID() mainheadId!` on the create DTO — mobile must send a valid MAINHEAD UUID.	If mobile's MAINHEAD list is empty for the pilot user (no MAINHEAD access), the dropdown is empty and submission fails 400. No SiteVisit row → admin sees 0 visits.
Non-ADMIN team-membership check at create time.	If pilot user is not in the team they selected, the create call throws Forbidden. No SiteVisit row → admin sees 0 visits.
New-pencawang GPS / Kod / Nama requirements.	Missing field → 400. No SiteVisit row.
Tenant-binding to mobile user.	Row is written with `tenantId = mobile.user.tenantId`. If the admin user is in a different tenant, the row exists but is invisible to the admin. Probe 1 in the 8-point pilot-state report distinguishes this.

All of these produce the observed pattern (Assets visible because assets attach to Substations independent of visits; Site Visits empty because no row was persisted in admin's tenant; Dashboard and Operations Board empty as downstream effects).